Closures in Depth

A **closure** is a function bundled together with references to the variables in the scope where it was *defined*, not where it is *called*. Every function in JavaScript is a closure, but the concept only becomes interesting when an inner function outlives the outer function that created it — and still reads and writes that outer function's local variables. Interviewers lean on closures because they test whether you truly understand lexical scope, variable lifetime, and the difference between a value and a live reference. Get closures right and you can reason about data privacy, callbacks, `this`, memory leaks, and half of the "what does this print?" traps in one shot.

Precise definition

Formally: a closure is the pairing of a function with its **lexical environment** — the set of variable bindings that were in scope when the function was created. When you define a function inside another function, the inner function captures the *variable itself* (the binding), not a snapshot of its value. So even after the outer function returns and its call frame is popped off the stack, any binding still reachable from a surviving inner function is kept alive on the heap.

The "how" is scope chain resolution. When code inside the inner function references a name, the engine looks it up in the inner scope, then the enclosing function scope, then outward to global — this chain is fixed at definition time (lexical), which is why the outer function having already returned is irrelevant.

JS Copy

```
function makeCounter() {
  let count = 0;                    // lives in makeCounter's environment
  return function increment() {
```

```
  count += 1;                    // reaches OUT to count via the scope cha
};
}

const next = makeCounter();
console.log(next()); // ⇒ 1
console.log(next()); // ⇒ 2
console.log(next()); // ⇒ 3
```

`makeCounter` has already returned by the time we call `next()`, yet `count` survives because `increment` still references it. Each call to `makeCounter()` creates a *fresh* environment, so two counters never share state:

JS Copy

```
const a = makeCounter();
const b = makeCounter();
console.log(a()); // ⇒ 1
console.log(a()); // ⇒ 2
console.log(b()); // ⇒ 1   (b has its own count)
```

Practical uses

Closures are not a party trick — they underpin a large chunk of real JavaScript patterns.

Data privacy / encapsulation (module pattern). Variables in the enclosing scope are unreachable from outside; only the methods you return can touch them. This is genuine private state, unlike a plain object property.

JS Copy

```
function createBankAccount(initial) {
  let balance = initial;                    // truly private
  return {
    deposit(n) { balance += n; return balance; },
    withdraw(n) {
      if (n > balance) throw new Error("Insufficient funds");
      balance -= n;
```

```
  };
}

const acct = createBankAccount(100);
acct.deposit(50);            // 150
console.log(acct.getBalance());   // ⇒ 150
console.log(acct.balance);        // ⇒ undefined  (no direct access)
```

Function factories. A function that returns configured functions, each closing over its own arguments.

JS Copy

```
function multiplier(factor) {
  return (n) ⇒ n * factor;
}
const double = multiplier(2);
const triple = multiplier(3);
console.log(double(5), triple(5)); // ⇒ 10 15
```

Memoization. The cache lives in the closure, private and persistent across calls.

JS Copy

```
function memoize(fn) {
  const cache = new Map();
  return function (n) {
    if (cache.has(n)) return cache.get(n);
    const result = fn(n);
    cache.set(n, result);
    return result;
  };
}
const slowSquare = (n) ⇒ n * n;
const fastSquare = memoize(slowSquare);
console.log(fastSquare(4)); // ⇒ 16  (computed)
console.log(fastSquare(4)); // ⇒ 16  (from cache)
```

JS Copy

```
function once(fn) {
  let called = false, result;
  return function (...args) {
    if (!called) { called = true; result = fn.apply(this, args); }
    return result;
  };
}
const init = once(() => { console.log("init!"); return 42; });
console.log(init()); // logs "init!" then => 42
console.log(init()); // => 42   (does NOT log again)
```

Event handlers capture surrounding state, so a handler can "remember" which item it belongs to without re-deriving it.

Debounce — delay running until input stops. The pending `timer` id is held in the closure between calls.

JS Copy

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
// Only the LAST call within `delay` ms actually fires.
const log = debounce((x) => console.log("fired:", x), 100);
log("a"); log("b"); log("c"); // after ~100ms => "fired: c"
```

Throttle is the same idea inverted — the closure remembers the last-run timestamp and drops calls that arrive too soon.

The loop-closure bug with `var`

the loop has finished and `i` is its final value.

JS Copy

```
const fns = [];
for (var i = 0; i < 3; i++) {
  fns.push(() => i);
}
console.log(fns.map((f) => f())); // => [ 3, 3, 3 ]
```

Fix 1 — `let`. `let` is block-scoped and the spec creates a *fresh binding per iteration*, so each closure captures its own `i`.

JS Copy

```
const fns = [];
for (let i = 0; i < 3; i++) {
  fns.push(() => i);
}
console.log(fns.map((f) => f())); // => [ 0, 1, 2 ]
```

Fix 2 — IIFE. Create a new scope each iteration by immediately invoking a function that captures the current value as a parameter.

JS Copy

```
const fns = [];
for (var i = 0; i < 3; i++) {
  (function (j) {
    fns.push(() => j);
  })(i);
}
console.log(fns.map((f) => f())); // => [ 0, 1, 2 ]
```

Fix 3 — bind / extra argument. Pass the current value in as a bound argument (or as an extra parameter to the callback), so it is fixed at bind time rather than read later.

```
const fns = [];
for (var i = 0; i < 3; i++) {
  fns.push(((j) => () => j).call(null, i)); // or someFn.bind(null, i)
}
console.log(fns.map((f) => f())); // => [ 0, 1, 2 ]
```

The classic real-world version is `setTimeout` in a `var` loop printing `3 3 3`; switching to `let` prints `0 1 2`.

Closures and memory

A closure keeps its enclosing environment alive as long as the closure itself is reachable. That is the whole point — and also where leaks come from. What stays alive is **every variable in scopes the closure can reach**, even ones it never uses, because the engine keeps the environment record, not a hand-picked subset (most engines optimize away truly unreferenced bindings, but you cannot rely on that).

JS Copy

```
function makeLeaky() {
  const big = new Array(1_000_000).fill("*"); // ~ large
  return function () { return "hi"; };          // never touches `big`
}
const f = makeLeaky();
// `big` may be retained because it lived in f's environment.
```

The fix is to null out large references you no longer need, or not capture them at all. The flip side: a closure is **garbage collected once nothing references the closure anymore**. When `f` above goes out of scope and is unreferenced, both `f` and its captured environment become eligible for collection. A common leak is attaching closures as long-lived event listeners or timers that transitively hold large DOM nodes or data — remove the listener (or `clearInterval`) to let the environment be freed.

JS Copy

```
// ...later, to allow GC of handler and anything it closed over:
// element.removeEventListener("click", handler);
handler = null;
```

Currying / partial application as closures

Currying — transforming `f(a, b, c)` into `f(a)(b)(c)` — is built entirely on closures: each returned function closes over the arguments gathered so far. Partial application (fixing some arguments up front) is the same mechanism. This is a full topic on its own; here is the shape so you recognize it as closures at work.

JS Copy

```
const add = (a) => (b) => (c) => a + b + c;
console.log(add(1)(2)(3)); // => 6

// Partial application via bind:
const add3 = (a, b, c) => a + b + c;
const add10 = add3.bind(null, 10);
console.log(add10(20, 30)); // => 60
```

Common interview questions

"Implement a counter that can't be tampered with." Keep the count in the closure and expose only controlled methods — there is no property to reassign from outside.

JS Copy

```
function Counter() {
  let count = 0;
  return {
    inc() { return ++count; },
    dec() { return --count; },
    value() { return count; },
  };
}
```

```
console.log(c.count);   // ⇒ undefined  (cannot be tampered with)
```

"What does this print?" (the `var` loop) — `[3, 3, 3]` (or `3 3 3` from `setTimeout`), because all closures share one function-scoped `i`. With `let`, `[0, 1, 2]`.

"What is a closure, precisely?" A function together with the lexical environment it was defined in; it retains access to that environment's bindings even after the outer function returns.

"Why does this counter not reset between calls?" Because `makeCounter()` returns *one* function closing over *one* `count`; calling that same returned function repeatedly mutates the same binding. Only calling `makeCounter()` again makes a new one.

"Write `once()` / `memoize()` / `debounce()`." All three store private state (`called`, a cache `Map`, a `timer` id) in an enclosing scope — the interviewer is checking that you reach for a closure rather than a global.

Gotchas / traps

- **`var` in loops** shares one binding — the `[3, 3, 3]` classic. `let` fixes it; an IIFE or a bound argument fixes it pre-ES6.
- **Shared vs fresh environment.** Two closures from the *same* `makeX()` call share state; from *different* calls they don't. Interviewers flip between these to see if you noticed.

JS Copy

```
function make() {
  let n = 0;
  return { a: () ⇒ ++n, b: () ⇒ ++n };
}
const m = make();
console.log(m.a()); // ⇒ 1
console.log(m.b()); // ⇒ 2   (a and b share n)
```

final `i`.

- **"Private" is real, but methods leak references.** If a privileged method *returns* the internal object/array directly, callers can mutate your private state through that reference. Return a copy if it must stay private.
- **Accidental retention.** A closure keeps its whole environment alive; capturing a huge array or a DOM node inside a long-lived handler is a memory leak. Detach listeners / clear timers / null references you no longer need.
- **Closures don't copy, they link.** Mutating the captured variable from anywhere (including the outer function after returning the closure) is visible to the closure — they share the one binding.

QUICK CHECK — no marks, just for you.

1. What does this log?

JAVASCRIPT Copy

```javascript
function makeCounter() {
  let count = 0;
  return function () {
    count += 1;
    return count;
  };
}
const a = makeCounter();
const b = makeCounter();
console.log(a(), a(), b(), a());
```

- ◯ 1 1 1 1
- ◯ 1 2 2 3
- ◯ 1 2 1 3
- ◯ 3 3 3 3

2. What does this log?

JAVASCRIPT Copy

```
const fns = [];
for (var i = 0; i < 3; i++) {
  fns.push(() => i);
}
console.log(fns.map((f) => f()));
```

- ◯ `[ 3, 3, 3 ]`
- ◯ `[ 0, 1, 2 ]`
- ◯ `[ 0, 0, 0 ]`
- ◯ `undefined`

Check answer

3. Which statement most precisely describes what a closure is?

- ◯ Any function that takes another function as an argument or that returns a function.
- ◯ A function that is copied along with a snapshot of its outer variables' values captured at definition time.
- ◯ A function whose scope chain is resolved based on where it is called rather than where it is defined.
- ◯ A function together with the lexical environment it was defined in, retaining access to that environment's bindings even after the outer function returns.

Previous
Execution Context & the Call Stack

Next
The this Keyword

LEARN MORE

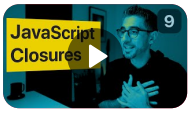
ARTICLES & DOCS

- Closures (MDN)
  MDN
- Closure (javascript.info)
  javascript.info

VIDEOS



- JavaScript Visualized — Closures
  YouTube



- Closures explained in depth
  YouTube



- Scope & closures, beginner → advanced
  YouTube

Hand-picked references for this lesson.